Research proposal — Sofiia Bakina

1. Motivation and method

Problem. Detecting whether a passage was in a model's pre-training corpus is usually attempted with heuristics over token likelihood. Zhang et al. replace the heuristic with a theoretical claim and derive a detector from it.

Mechanism. Maximum-likelihood training pushes training tokens towards being *local maxima* of the modelled distribution along each input dimension. The right question is therefore not whether a token has high absolute probability, but whether it is a **mode** of the conditional distribution. Min-K%++ operationalises this by standardising each token's log-probability against the mean and standard deviation of the model's next-token distribution over the whole vocabulary,

$$\text{Min-K\%++}(x_{<t}, x_t) = \frac{\log p(x_t \mid x_{<t}) - \mu_{\cdot \mid x_{<t}}}{\sigma_{\cdot \mid x_{<t}}}$$

and then averaging the lowest k% of these z-scores over the sequence. Both μ and σ are computed analytically from logits the model already produces, so the method adds **no inference cost** over its predecessor.

Why this paper. The improvement is principled rather than tuned: the calibration is derived from an argument about training dynamics, and the local-maximum prediction is verified empirically — including on a diffusion model, which is stronger evidence than a leaderboard gain. The paper is also honest about its own decomposition: an ablation separates the contributions of the mean and the variance terms, so the mechanism is inspectable. And it surfaces the limitation I find most interesting — performance still depends on input length, and short inputs remain the hardest case for every method in the field.

Result chosen for reproduction. Table 1 — AUROC on WikiMIA. The headline claim is that in the original setting Min-K%++ beats the runner-up Min-K% on average by **10.5, 12.3 and 6.2 AUROC points** at input lengths 32, 64 and 128. I reproduce the two cheapest model columns at length 32:

Model	Min-K%	Min-K%++	Gap
Mamba-1.4B	63.2	**66.8**	+3.6
Pythia-6.9B	66.3	**70.3**	+4.0

together with the Loss, Zlib and Lowercase baselines from the same table, so that the **ordering** of methods is tested and not merely the gap.

Secondary claim, and the hook for my extension. The paper reports that Min-K%++ degrades far less than Min-K% as inputs shorten: on Mamba-1.4B, Min-K% scores {62.2, 66.8} at lengths {64, 128}, a 4.6-point drop for the shorter input, while Min-K%++ scores {67.2, 68.8} — a drop of only 1.6. The length dependence is **reduced but not removed**, and that residual is what I propose to attack.

2. Critical analysis

Documented by the authors.

#	Limitation	Consequence
D1	k remains a hyperparameter; the ablation shows robustness — 2.7% between best and worst on LLaMA-13B, against 4.4% for Min-K% — but a value must still be chosen	Robust is not invariant; the choice is still unexplained, and the comparison is a claim about *relative* stability rather than about correctness at any particular k
D2	μ and σ require the **full next-token distribution**, not just the target token's probability	A strictly stronger access assumption than Min-K%; rules out most commercial APIs
D3	On MIMIR the gains are small and can be negative, improving with scale: {−0.2, +0.5, +1.1, +1.8, +2.8} across Pythia 160M–12B	The advantage is not universal; on a harder benchmark it nearly vanishes at small scale
D4	Performance still depends on input length	The dependence is reported and reduced, never eliminated
D5	The paraphrased WikiMIA split is the authors' own ChatGPT reconstruction, the original never having been released	Paraphrased results are not strictly comparable with Shi et al.'s
D6	**The advantage is model-dependent on WikiMIA too.** Computed from Table 1, the Min-K%++ minus Min-K% gap averages **+3.7** points over Mamba-1.4B and Pythia-6.9B against **+13.6** over the three LLaMA columns	The headline averages (+10.5 / +12.3 / +6.2) are carried by LLaMA. The two models inside my budget are the two where the method's advantage is smallest, which sets what this reproduction can and cannot establish. This is arithmetic on their table, not a conjecture

Measured in a pilot for this proposal — including the target cell itself. A faithful port of the authors' `run.py` was run over all four WikiMIA splits, and AUROC uncertainty — a property of the sample, not of the model — was estimated with a paired bootstrap over items (2000 resamples).

The Mamba-1.4B column of Table 1 at length 32 has already been reproduced, on CPU, in 30 minutes for all 776 items:

Method	This pilot	Table 1	Difference
Loss	61.0	61.0	0.0
Zlib	61.9	61.9	0.0
Min-K% (best k)	63.3	63.2	+0.1
Min-K%++ (best k)	**66.8**	**66.8**	**0.0**
Min-K%++ − Min-K%	**+3.6**	**+3.6**	**0.0**

Ref, Lowercase and Neighbor need separate passes and are not included. What this establishes before the project starts: the scoring path is correct, the cheaper of the two model columns is done, and the 14 October checkpoint is a measurement rather than an open question. It also turns the interval below from an estimate into a measurement on this model.

#	Measurement	Consequence
M1	95% CI half-width of a **single** AUROC: **±4.0** points at length 32 (n = 776), ±4.8 at 64 (n = 542), ±7.0 at 128 (n = 250), ±13.4 at 256 (n = 82)	A reproduction landing 3 points from a published cell has reproduced it. This is the quantitative yardstick for the discrepancy diagnosis in §5
M2	95% CI half-width of the **paired** Min-K%++ − Min-K% gap: **±3.2, measured on Mamba-1.4B itself** — the reproduced gap is +3.6 with a 95% interval of [+0.4, +6.8] and P(gap > 0) = 0.989. Estimated at ±3.3 for length 64, ±6.2 at 128, ±12.5 at 256	The advantage on this model clears zero, but only just: the lower bound is +0.4 of a +3.6 point estimate. With D6 the +4.0 gap on Pythia-6.9B sits in the same regime, and at length 128 the published +2.0 and +1.2 fall inside the interval entirely. Pairing the resampling is what buys ±3.2 instead of ±3.9, so it is not optional
M3	Token counts **vary within a split**: "length 32" spans 33–130 tokens, median 42, sd 9.2, because 32 is a word count rather than a token count. Within-split coefficient of variation: 0.22, 0.13, 0.10, 0.07	Direct support for C1, and stronger than the between-split evidence: the statistic's length dependence is present *inside* every split, largest in the split the paper calls hardest, and nowhere reported
M4	Median token count is balanced across classes (42 against 43 at length 32)	The within-split variation is a nuisance rather than a label leak, which is what makes correcting for it a fair change and not a shortcut

My conjectures, requiring investigation.

#	Conjecture	Why it matters, and what evidence is missing
C1	**The min-k% mean is a length-dependent statistic.** Keeping a *fraction* of tokens means the selected set grows with sequence length, and the expected minimum of a larger sample is more extreme	The score distribution therefore shifts with length, so scores are not comparable across lengths and a single threshold cannot transfer. The paper measures the length effect but never attributes it, and M3 shows the variation is present within a split as well as between splits. **This is what I test.**
C2	Averaging over a whole sequence dilutes localized membership	A long passage containing one memorized span inside novel text is scored as mostly-novel; no experiment isolates a span inside a longer input
C3	Equal weighting inside the min-k% set discards magnitude	A token at $z = -4$ counts the same as one at $z = -1.1$; the theory says depth below the mode is meaningful
C4	The theory is per-token, the statistic is a sequence mean	The step from "training tokens are local maxima" to "average the lowest k% of z-scores" is heuristic, not derived
C5	Length splits differ in content, not only in length	WikiMIA's 32/64/128 splits are separate texts, so the length effect is confounded with content; no experiment truncates fixed content to vary length alone
C6	No interval is reported	Single-run numbers on a fixed benchmark subsample, with no resampling CI on any AUROC. M1 and M2 size what that omission hides: at this n, several of the paper's own reported gaps are not separable from zero

3. Ten potential improvements

Each changes **how per-token scores combine into a sequence score**, not the dataset and not the per-token score function. Full matrix in **Appendix A**.

1. **Sliding-window aggregation** — score overlapping fixed-width windows and pool by the maximum instead of averaging the whole sequence. *(C2, C1)*
2. **Fixed-count selection** — keep the m lowest-z tokens rather than the lowest k%, decoupling set size from length. *(C1)*
3. **Length-normalised aggregation** — standardise the min-k% mean against its expected value under a length-matched null. *(C1)*
4. **Magnitude-weighted aggregation** — weight each selected token by |z| so depth below the mode counts. *(C3)*
5. **Order-statistic aggregation** — use a fixed quantile of the z-distribution rather than the mean of a tail. *(C3, C4)*
6. **Chunk-and-vote** — split into fixed chunks, score each, aggregate by a high quantile of chunk scores. *(C2)*
7. **Length-conditional calibration** — fit a per-length calibration map so one threshold transfers across splits. *(C1)*
8. **Position-weighted aggregation** — downweight early tokens whose z-scores are noisy for lack of context. *(C4)*
9. **Two-sided aggregation** — combine the low tail with the high tail rather than using the low tail alone. *(C4)*
10. **Early-termination scoring** — stop once the running statistic is decisive, for cheaper scoring of long documents. *(efficiency; D2 makes each call expensive)*

4. Selected improvement

Rank	Idea	Why here
1	**#1 Sliding-window aggregation**	Attacks C1 and C2 together, and makes a sharp directional prediction — the gain should grow with input length, which is testable against a number the paper already reports
2	#2 Fixed-count selection	The most direct fix for C1 alone, and computable from the same cached scores — hence the fallback
3	#4 Magnitude weighting	Cheap and theoretically motivated, but likely a smaller effect than either of the above

Selected: sliding-window aggregation with max pooling. Replace the whole-sequence min-k% mean with: slide a window of w tokens with stride s over the sequence, compute the Min-K%++ statistic within each window, and take the sequence score to be the **maximum** (or a high quantile) over windows. For sequences shorter than w this reduces exactly to the original method, so the baseline is a special case rather than a separate code path.

Boundary with adjacent work. S06 is working on the *predecessor* paper along a different axis — which tokens enter the selected set, corrected by an external corpus-frequency prior. My change leaves both the per-token score function and the selection criterion untouched and alters only the **aggregation geometry**. The two are separable, and the report will state the boundary.

Falsifiable hypothesis (H1). The residual length dependence of Min-K%++ is an artifact of aggregating over the whole sequence. Sliding-window max pooling will raise AUROC relative to Min-K%++, and **the gain will increase with input length** — larger at 128 than at 32 — while the 64→128 gap on Mamba-1.4B shrinks below the reported 1.6 points. **Null (H0).** Windowed and

whole-sequence aggregation give equal AUROC within a bootstrap interval at every length, and the length gap is unchanged.

Expected benefit. If H1 holds, a strictly more general aggregation rule that contains the original as a limiting case, with the gain concentrated on long inputs where dilution should bite — and a cleaner account of *why* Min-K%++ is more length-stable than Min-K%. If it does not, that is evidence that the length effect is a property of the content in WikiMIA's splits rather than of the statistic, which is C5 and is itself worth reporting.

Implementation steps. (1) Re-implement the Min-K%++ scorer from Equations 3–4 so the aggregation step is mine and parameterisable, calling the authors' data loaders unmodified. (2) Cache **per-token z-scores** for every WikiMIA item once — every aggregation variant is then a CPU-only post-processing step. (3) Reproduce the two Table 1 columns: the 14 October checkpoint. (4) Sweep w and s and evaluate windowed against whole-sequence aggregation at each length. (5) Test the length prediction directly by **truncating fixed content** to 32/64/128 tokens, which removes the C5 confound the benchmark's own splits cannot.

Fallback. If windowing gives nothing, I pivot to **#2, fixed-count selection**, computable **from the same cached z-scores** and therefore needing no new GPU time — chosen as the fallback for exactly that reason. If instead the *baseline* fails to reproduce at step 3, the project becomes a diagnosis of that discrepancy, reported as such.

5. Evaluation and delivery plan

Datasets and splits. WikiMIA (`swj0419/WikiMIA`) at lengths 32, 64 and 128, plus the authors' paraphrased and perturbed reconstruction (`zjysteven/WikiMIA_paraphrased_perturbed`); per D5 the paraphrased numbers are reported as an approximate replication rather than compared directly with Shi et al. A **truncation split** is derived from the 128-token items by cutting the same content to 64 and 32 tokens, which varies length while holding content fixed and is the only clean test of C5. Window size w, stride s and any k variant are fitted on a held-out 20% of items, disjoint from the evaluation set; **nothing is tuned on the test split.**

Baseline. Min-K%++ at the released defaults on `state-spaces/mamba-1.4b-hf` and `EleutherAI/pythia-6.9b`, together with Loss, Zlib, Lowercase and Min-K% from the same forward pass, reproducing the two Table 1 columns. Whole-sequence aggregation is the $w = \infty$ case of the variant, so baseline and extension share one code path.

Metrics. AUROC as the primary, matching the paper; **TPR@5%FPR**, which the authors also report; AUROC broken out by length; and two mechanism checks — the **64→128 length gap**, which H1 predicts shrinks, and the position of the winning window, which should be non-uniform if dilution is real. Every comparison carries a **paired** bootstrap interval and the minimum detectable effect at that n, because per M2 the quantity of interest is a difference between two scores on the same items, and resampling them jointly is what makes a 3-point effect resolvable at all.

Matched compute. Baseline and every variant read the *same cached z-scores*, so compute is matched exactly by construction: one forward pass per item serves all conditions. Post-processing wall-clock is reported per variant, and the Neighbor baseline — the only method needing extra generation — is reported with its cost stated separately.

Ablations. (a) window size w and stride s, including $w = \infty$ which recovers the baseline exactly; (b) pooling function — max, 90th percentile, mean-of-top-3 — to show the result is not an artifact of the max alone; (c) length, 32/64/128, on both the native splits and the truncation split, which is the discriminating test; (d) $k \in \{10, 20, 30\}$, to check that windowing and k do not interact.

Seed strategy. The scorer is deterministic given a model, so variance enters through the benchmark subsample rather than the method, and the usual three-seed requirement does not apply to it in the

usual sense. I report **2000-sample paired bootstrap intervals** over WikiMIA items for every AUROC and every difference, seeded at 0, with resampling indices shared across methods so that gaps stay paired; the pilot fixes the expected half-widths at ±4.0 / ±4.8 / ±7.0 per split for a single AUROC and ±3.3 / ±3.3 / ±6.2 for a paired difference (M1, M2), so the study's resolution is known in advance rather than discovered afterwards. The Neighbor baseline — the one stochastic component — runs with three seeds (42, 43, 44). This substitution is stated in the report rather than left to read as an omission.

Resources, with the download and memory figures measured rather than estimated. One GPU with ≥ 16 GB. Mamba-1.4B is ≈ 2.8 GB in fp16 and Pythia-6.9B ≈ 13.8 GB, fitting on a 16 GB card with 8-bit loading as a documented fallback; nothing is fine-tuned. Two practical facts were checked directly and are not what the fp16 footprints suggest: **Mamba-1.4B is 5.49 GB to download**, two fp32 safetensors shards rather than the 2.8 GB its fp16 size implies, and Pythia-6.9B is 13.85 GB, so it does not fit in 13.7 GB of system RAM and has no CPU fallback. Mamba does: a full pass over all 776 items of the length-32 split takes two minutes for a 160M model on CPU, extrapolating to one or two hours for Mamba-1.4B, so **half the reproduction needs no GPU at all** and hosted quota is reserved for Pythia-6.9B. `mamba-ssm` and `causal-conv1d` appear in the authors' requirements and build poorly on Windows; `MambaForCausalLM` was verified to load through `transformers` without them, so they are not a blocking dependency. Estimated **1–3 GPU-hours** for the cached pass across both models, three lengths and both settings; the Neighbor baseline adds roughly 30–60 minutes. Caching is what makes the compressed Group B schedule workable: after 14 October the GPU is essentially done and the rest is analysis.

Weekly tasks.

Week	Dates	Task
6	7–13 Oct	Scorer re-implemented from Eq. 3–4; z-score cache built; truncation split derived
7	14–20 Oct	**Baseline checkpoint (14 Oct):** two Table 1 columns reproduced; discrepancy diagnosis
8	21–27 Oct	w/s sweep; windowed vs whole-sequence at all lengths; ablations (a)–(c); progress report (21 Oct)
9	28 Oct–3 Nov	**Review package (28 Oct)**; ablation (d); feedback 1 Nov; response and slides 3 Nov
10	4 Nov	**Defence.** Final archive 7 Nov
11	5–10 Nov	Review of S13 (due 8 Nov)
12	11–17 Nov	Review of S19 (due 15 Nov)

Review deadlines follow the author's group (§7.1), so both of my reviews fall *after* my own defence — which is why the experimental work is front-loaded into weeks 6–8.

Intended presentation group. Group B, Week 10, 4 November 2026 — as published in §9. I am not requesting a change: the project is compute-light and front-loaded, so the early slot is an advantage rather than a risk.

References

Zhang, J., Sun, J., Yeats, E., Ouyang, Y., Kuo, M., Zhang, J., Yang, H. F., & Li, H. (2025). Min-K%++: Improved Baseline for Pre-Training Data Detection from Large Language Models. *ICLR 2025*. https://openreview.net/forum?id=ZGkfoufDaU

Shi, W., et al. (2024). Detecting Pretraining Data from Large Language Models. *ICLR 2024*. https://openreview.net/forum?id=zWqr3MQuNs

Duan, M., et al. (2024). Do Membership Inference Attacks Work on Large Language Models? (MIMIR). *COLM 2024*.

Biderman, S., et al. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *ICML 2023*.

Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.

Code: https://github.com/zjysteven/mink-plus-plus

Appendix A — Ten-improvement matrix

#	Limitation addressed	Proposed change	Why it could help	Experiment / metric	Effort & compute
1	**C1, C2** — whole-sequence averaging is length-dependent and dilutes localized signal	Score overlapping windows of width w, stride s, and pool by max or a high quantile; w = ∞ recovers the baseline	A memorized span raises one window's score regardless of how much novel text surrounds it, and the statistic no longer scales with sequence length	AUROC vs baseline per length; the 64→128 gap, predicted to shrink; position of the winning window	Low code, **no extra GPU** — post-processing on cached z-scores
2	**C1** — keeping a fraction ties set size to length	Keep the m lowest-z tokens instead of the lowest k%	Fixes the selected-set size, so the expected extremeness of the tail no longer grows with n	AUROC vs length for fixed m against fixed k; slope of AUROC in length	Low code, no extra GPU
3	**C1** — scores are not comparable across lengths	Standardise the min-k% mean by its expected value under a length-matched null (shuffled or random text of equal length)	Puts every length on a common scale, so one threshold transfers	Score-distribution overlap across lengths; threshold transfer accuracy	Low code, small extra cached pass for the null
4	**C3** — equal weighting discards depth below the mode	Weight each selected token by |z| when averaging	The theory says distance below the mode is meaningful; equal weighting throws that away	AUROC weighted vs unweighted; sensitivity to the weighting exponent	Low code, no extra GPU
5	**C3, C4** — the mean of a tail is an arbitrary summary	Use a fixed order statistic, e.g. the 10th percentile z-score, as the sequence score	An order statistic is robust to how many tokens fall in the tail, which is exactly what varies with length	AUROC across quantiles; comparison with the min-k% mean	Low code, no extra GPU
6	**C2** — one memorized span is averaged away	Split into fixed chunks, score each, and aggregate by a high quantile of chunk scores	A coarser, cheaper version of windowing that needs no overlap bookkeeping	AUROC vs chunk count; agreement with the windowed variant	Low code, no extra GPU
7	**C1** — a single threshold cannot serve all lengths	Fit a calibration map per length on held-out data	Makes the detector deployable at a stated false-positive rate across mixed-length inputs	TPR@5%FPR after calibration; threshold transfer between length splits	Low code, no extra GPU
8	**C4** — early tokens have unreliable z-scores	Downweight the first n tokens in the aggregate	Reduces variance contributed by positions with almost no conditioning context	AUROC vs n; per-position contribution profile	Low code, no extra GPU

#	Limitation addressed	Proposed change	Why it could help	Experiment / metric	Effort & compute
9	**C4** — only the low tail is used	Combine the low tail with the high tail, treating unusually mode-like tokens as evidence too	The local-maximum theory predicts training tokens sit *at* modes, which is a high-side signal the one-sided statistic ignores	AUROC two-sided vs one-sided; per-tail ablation	Low code, no extra GPU
10	**D2** — each call needs the full vocabulary distribution, so scoring is expensive	Terminate scoring once the running statistic is decisive against a calibrated boundary	Cuts cost on long documents, which matters because μ and σ make every token expensive	Agreement with full scoring vs fraction of tokens processed	Medium code, **saves** compute